

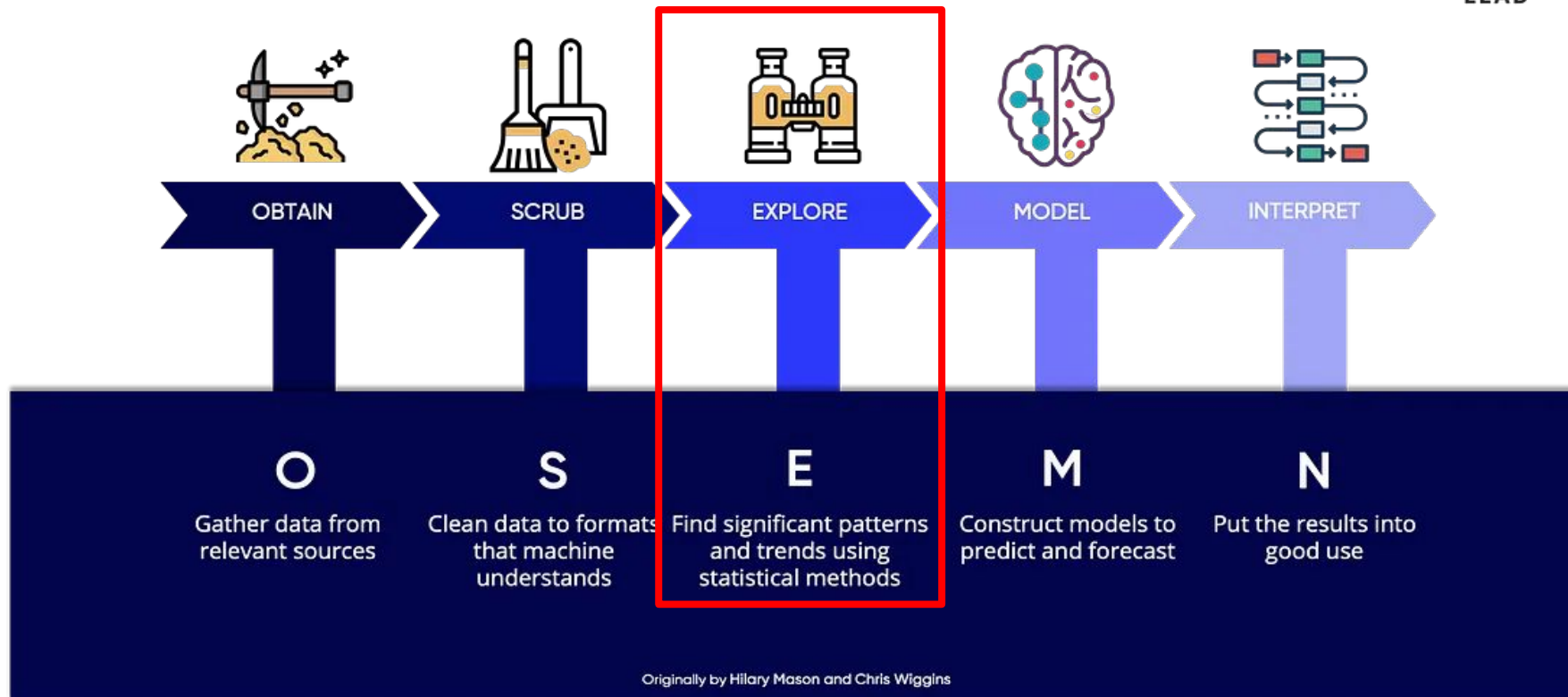
GCI World

Visualizing Data Using Matplotlib



**Focus of
this lecture**

Data Science Process



Reference:

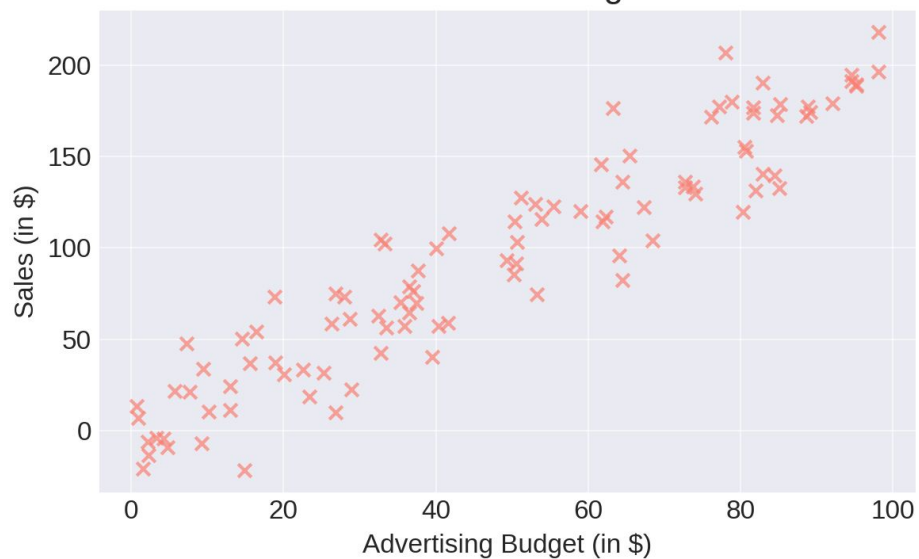
<https://towardsdatascience.com/5-steps-of-a-data-science-project-lifecycle-26c50372b492>

- Exploratory Data Analysis (EDA): An iterative process combining analysis and visualization to understand data.
- The purpose of EDA:
 - Identify patterns or biases from data
 - Form and test hypotheses
- Key points of EDA:
 - Quick and exploratory visualizations
 - Visualize to communicate

Examples of EDA

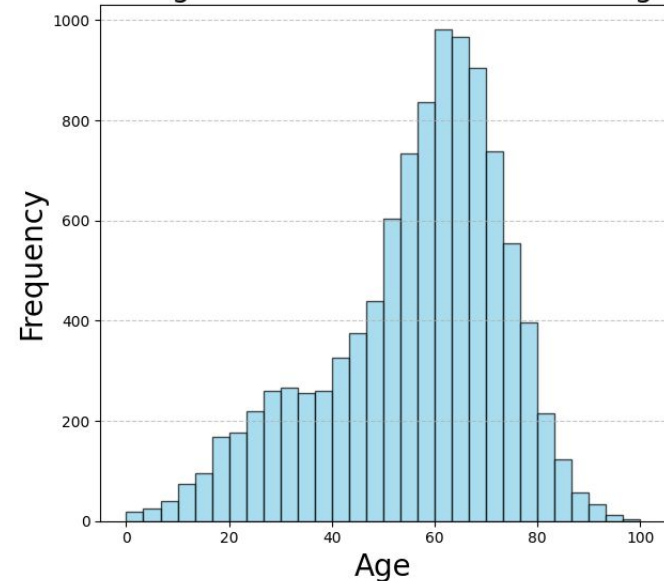


Scatter Plot: Advertising vs Sales



Advertising budget and sales seem to have a correlation.

Histogram: Skewed Distribution of Age



This dataset seems to be skewed toward older age groups.

Data visualization in Python mainly uses
a library called **Matplotlib**.



matplotlib

matplotlib.pyplot

- A **simplified version** of matplotlib
- Suited for trial-and-error visualization
- May be restricted when you want to create extremely detailed graphs.



Import the module as follows

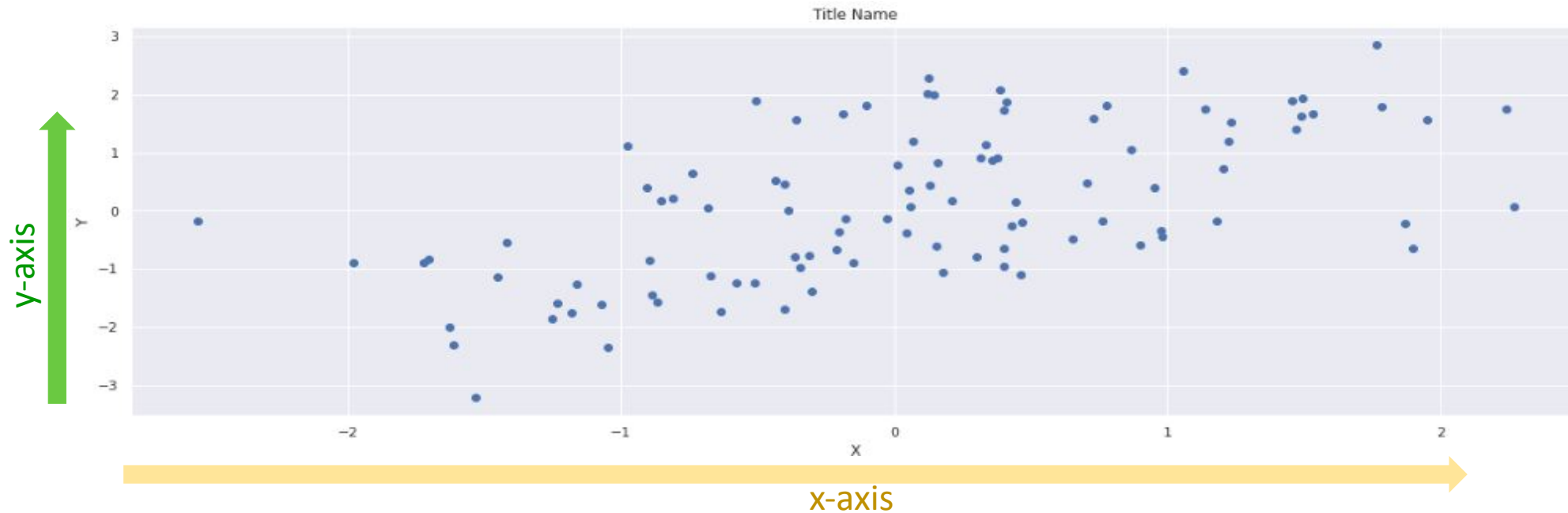
```
import matplotlib.pyplot as plt
```

Customizing graphs (3): Changing the size of the graph



`plt.figure(figsize={x-axis length}, {y-axis length})`

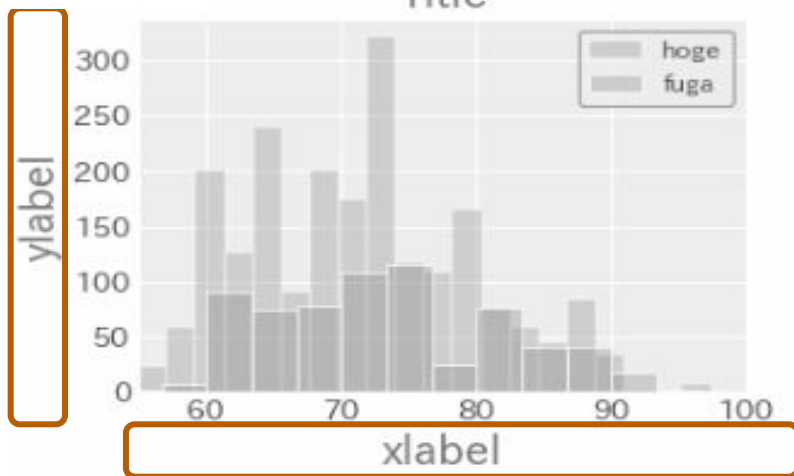
Example: `plt.figure(figsize=(20, 6))`



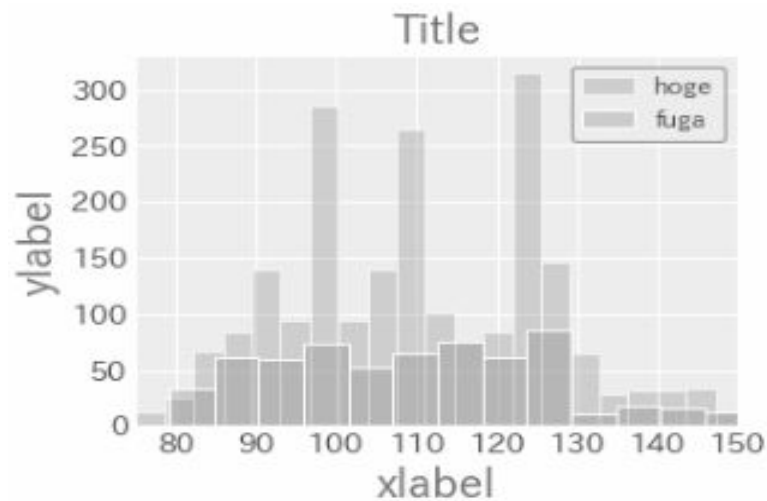
figure

axes

axis



axes



Graphs with multiple axes

Although introduced earlier in the context of specifying the graph size, the original function of this function is to “create a new Figure”

(If only one Figure is to be created, this function can be omitted.)

Function to create multiple Axes in Figure:
Three arguments are lined up in parentheses. The meanings are as follows

`plt.subplot( 2, 1, 2 )`

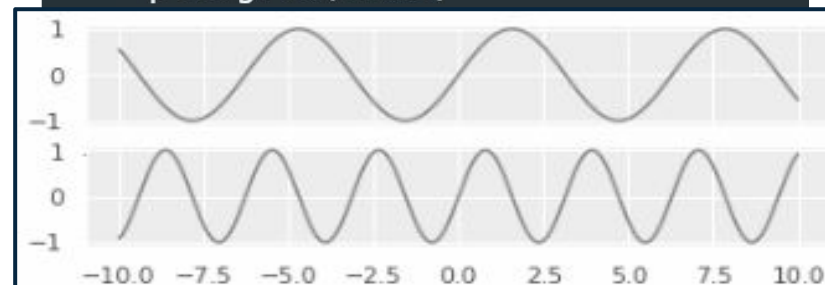
2nd axes of a figure with **2** rows and **1** column

```
# Specify graph size
(1) plt.figure(figsize=(20, 6))

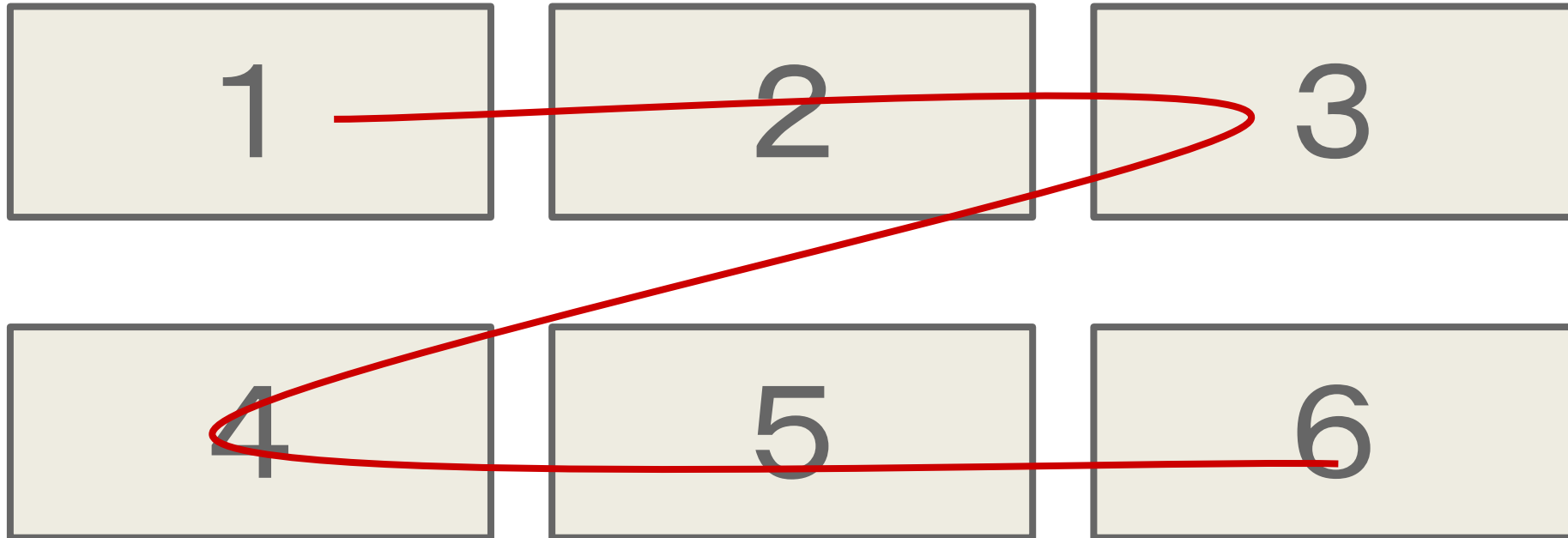
# The first of 2 rows and
# 1 column graphs
(2) plt.subplot(2, 1, 1)
x = np.linspace(-10, 10, 100)
plt.plot(x, np.sin(x))

# The second of 2 rows and
# 1 column graphs
(3) plt.subplot(2, 1, 2)
x = np.linspace(-10, 10, 1000)
plt.plot(x, np.sin(2*x))

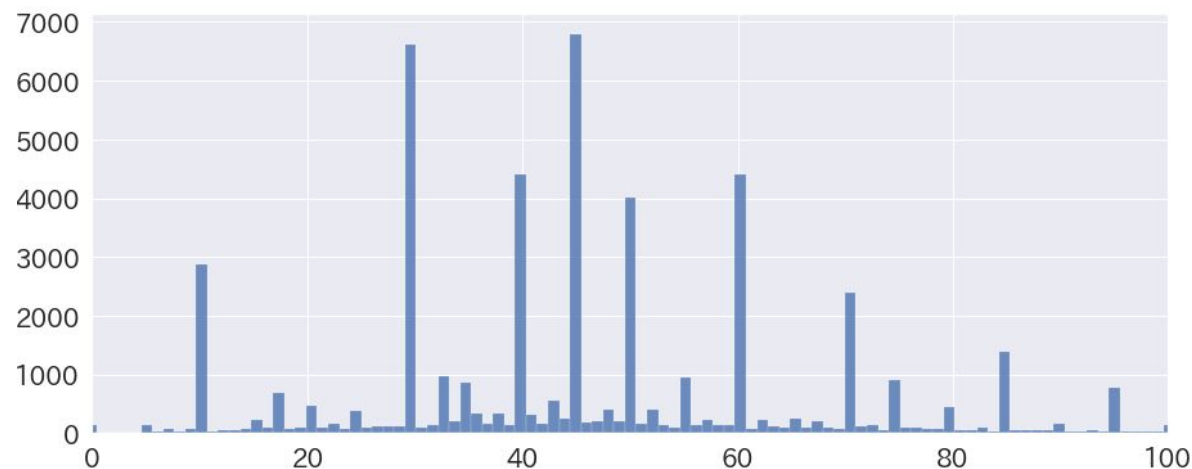
plt.grid(True)
```



`plt.subplot(2, 3, n)`

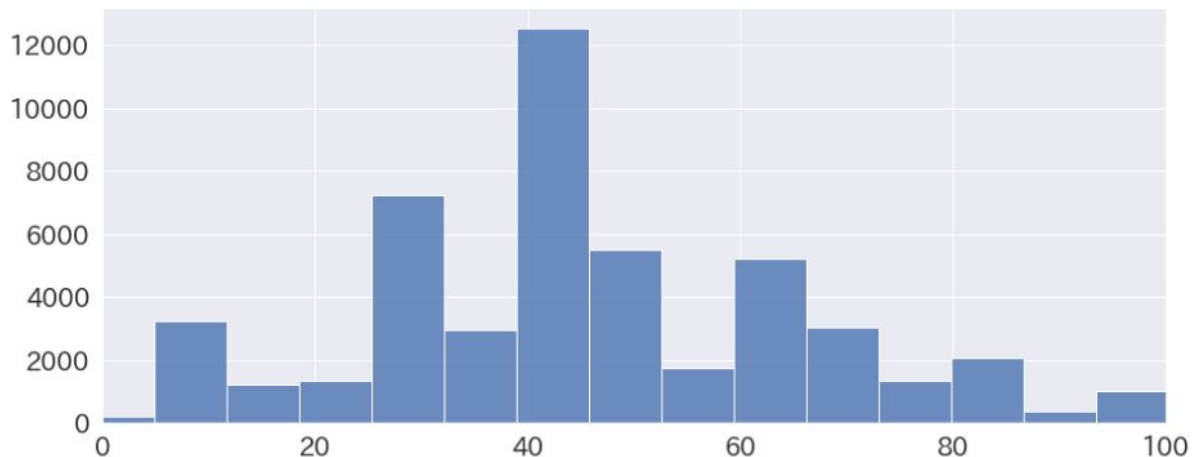


Customizing histograms: bins and range



When using smaller bins

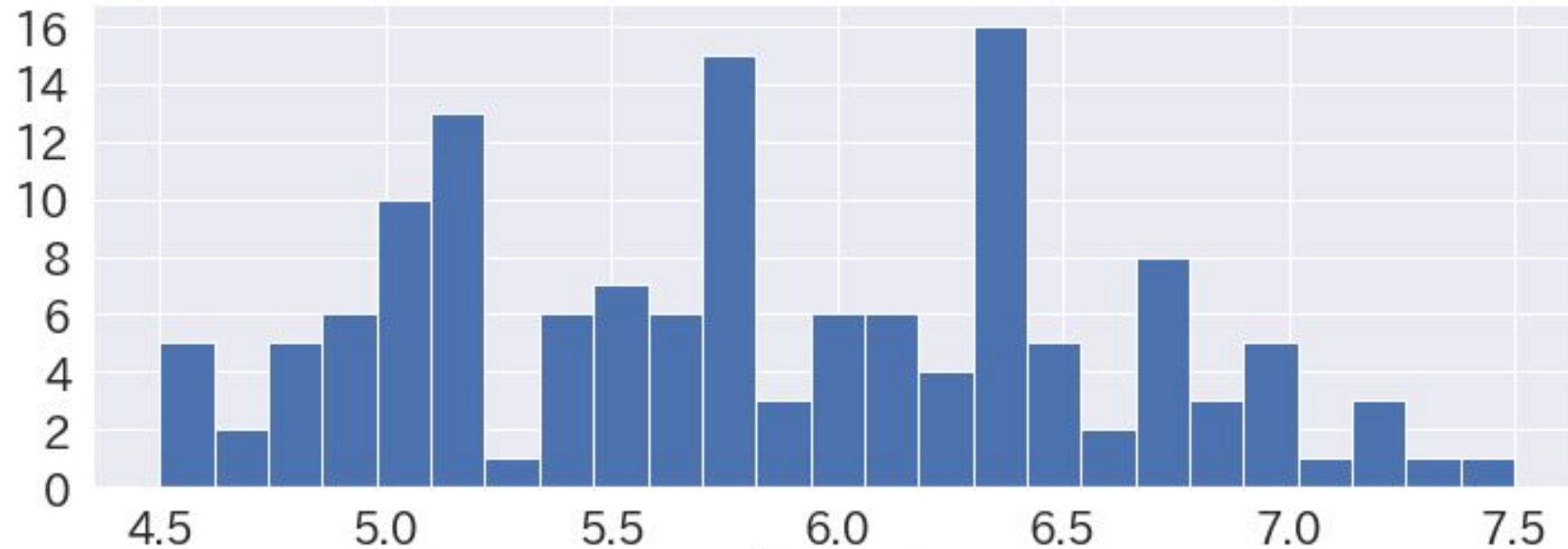
- It may be possible to find sections of the distribution that are locally concentrated
- It is difficult to grasp the overall trend



When using larger bins

- It is easy to grasp the overall trend
- Specific local distributions are not visible

Customizing histograms: bins and range



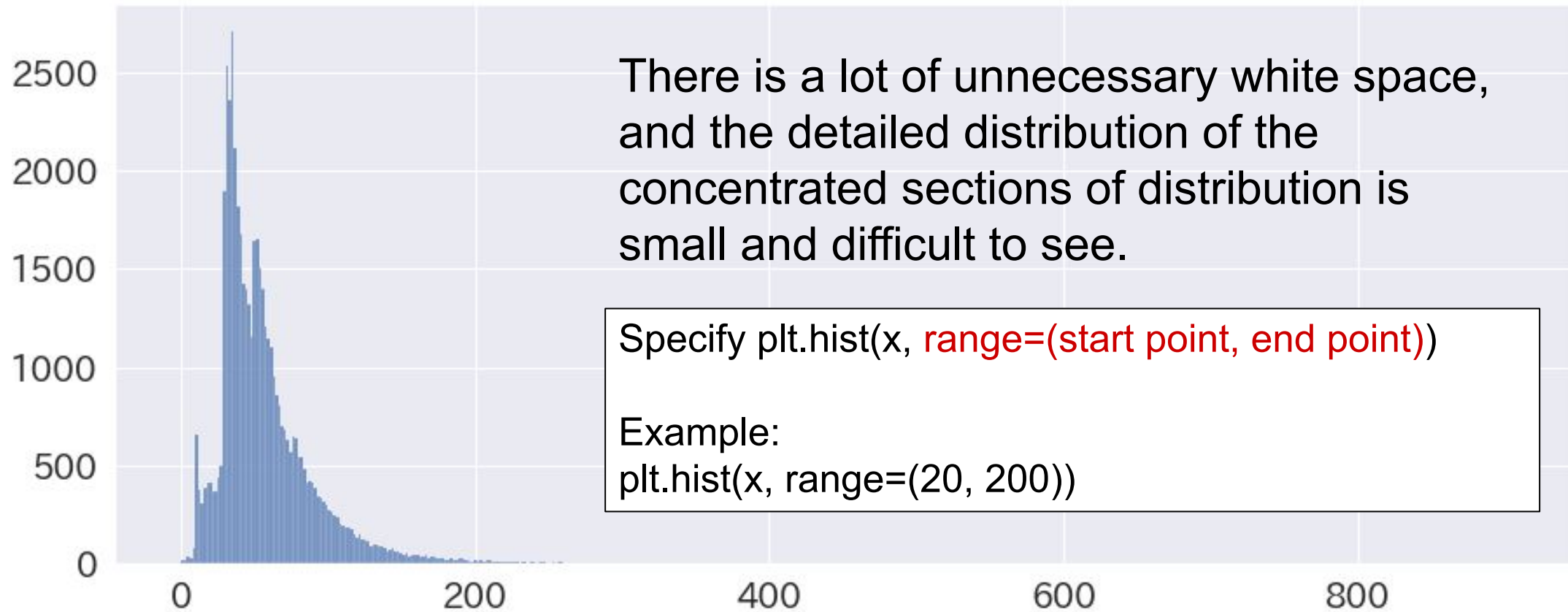
Pass an integer value in the argument “bins” to specify how many times to cut the interval.

Specify `plt.hist(x, bins={Number of bin})`

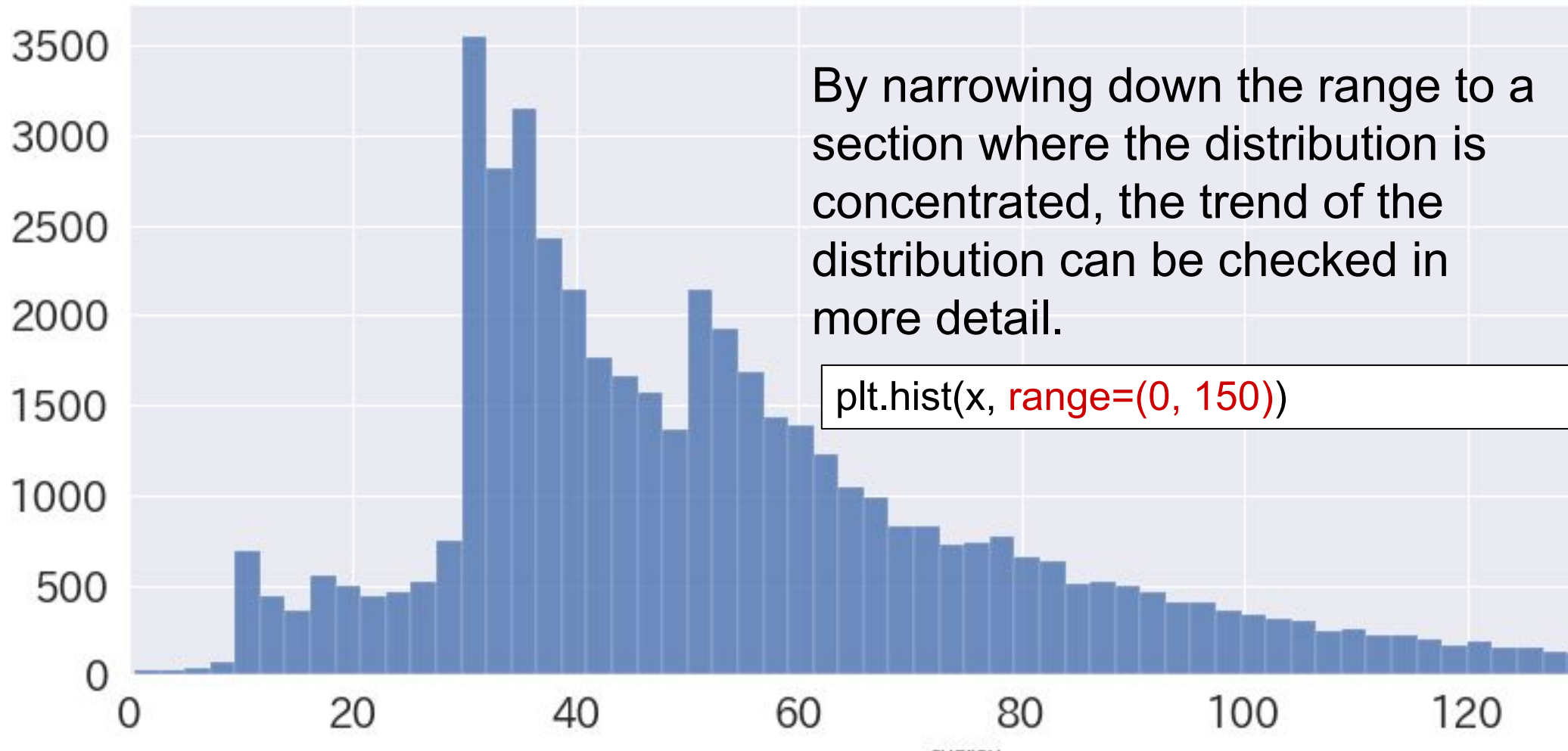
Example:

```
plt.hist(x, bins=25)
```

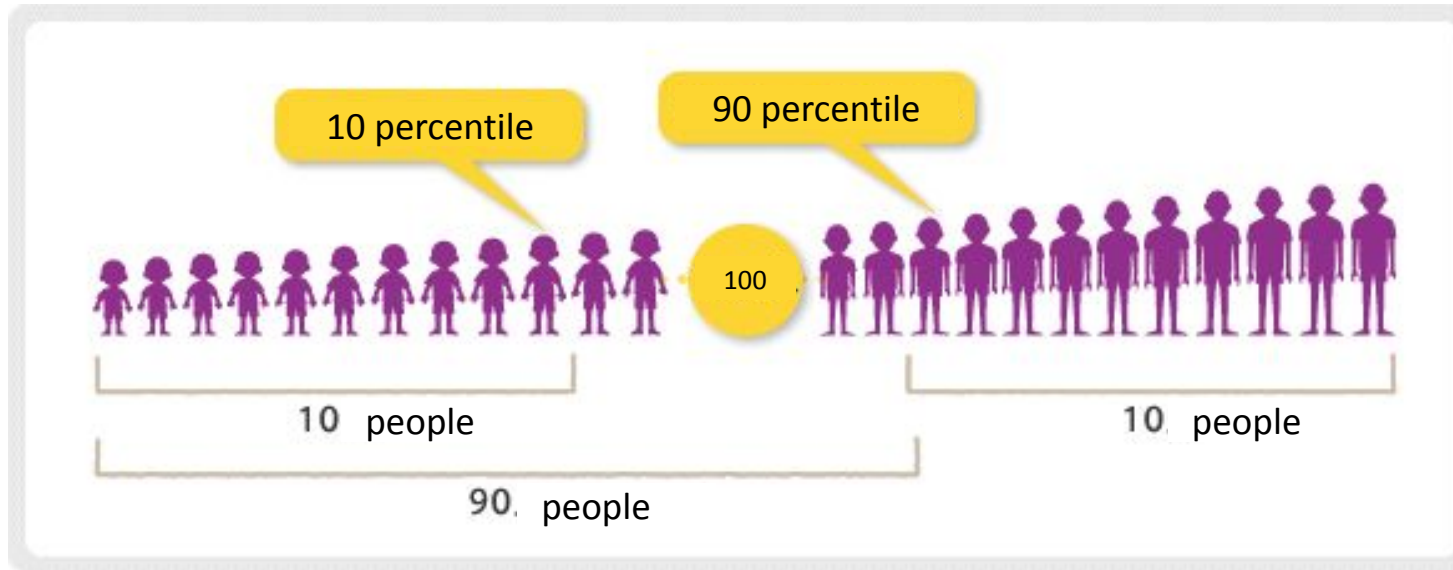
Customizing histograms: bins and range



Customizing histograms: bins and range



Understanding Percentiles and Summary Statistics



```
[ ] # summary statistics
student_data_math['absences'].describe()
```

count	395.000000
mean	5.708861
std	8.003096
min	0.000000
25%	0.000000
50%	4.000000
75%	8.000000
max	75.000000
Name: absences, dtype: float64	

The value that comes to the nth percentile counting from the smallest to the highest when the total is 100.

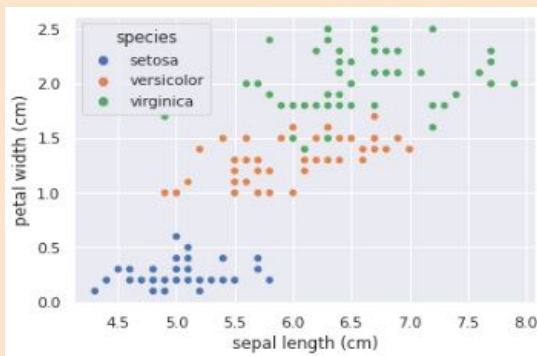
Reference URL:

http://ghw.pfizer.co.jp/comedical/evaluation/images/img_relation_01.gif

Pandas' `.describe()` method allows for batch calculations along with other statistics.

Quantitative data

Numerical data that can be added, subtracted, etc.
(price, time, age, etc.)



Includes both continuous and discrete values

Categorical data

Data that expresses categories or characteristics
(Gender, category, etc.)

address	famsize	Pstatus	Medu	Fedu
U	GT3	A	4	4
U	GT3	T	I	I
U	LE3	T	I	I

Medu (mother's education category)" is a numerical value, but it has no analytical meaning.

Creating different graphs: pie charts

【How to use】

Use `plt.pie()` for a **pie chart**
Pass the ordered list of elements to each function.

【Main arguments】

x

Value of each element

label

Label to display for each element

colors

The color of each element

explode

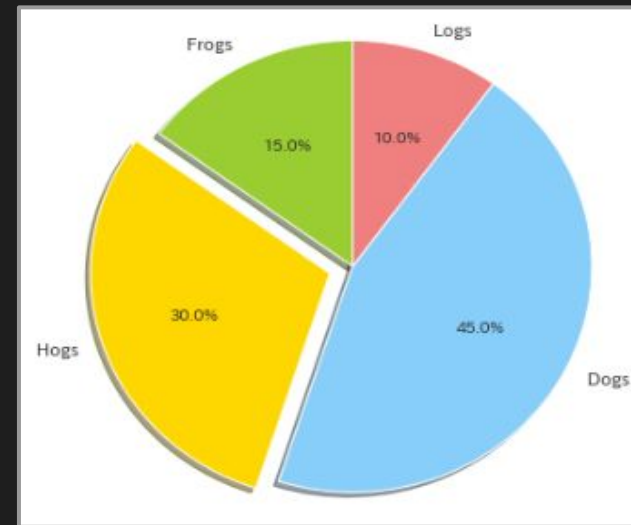
The value of how far away from the center each element should be displayed.

startangle

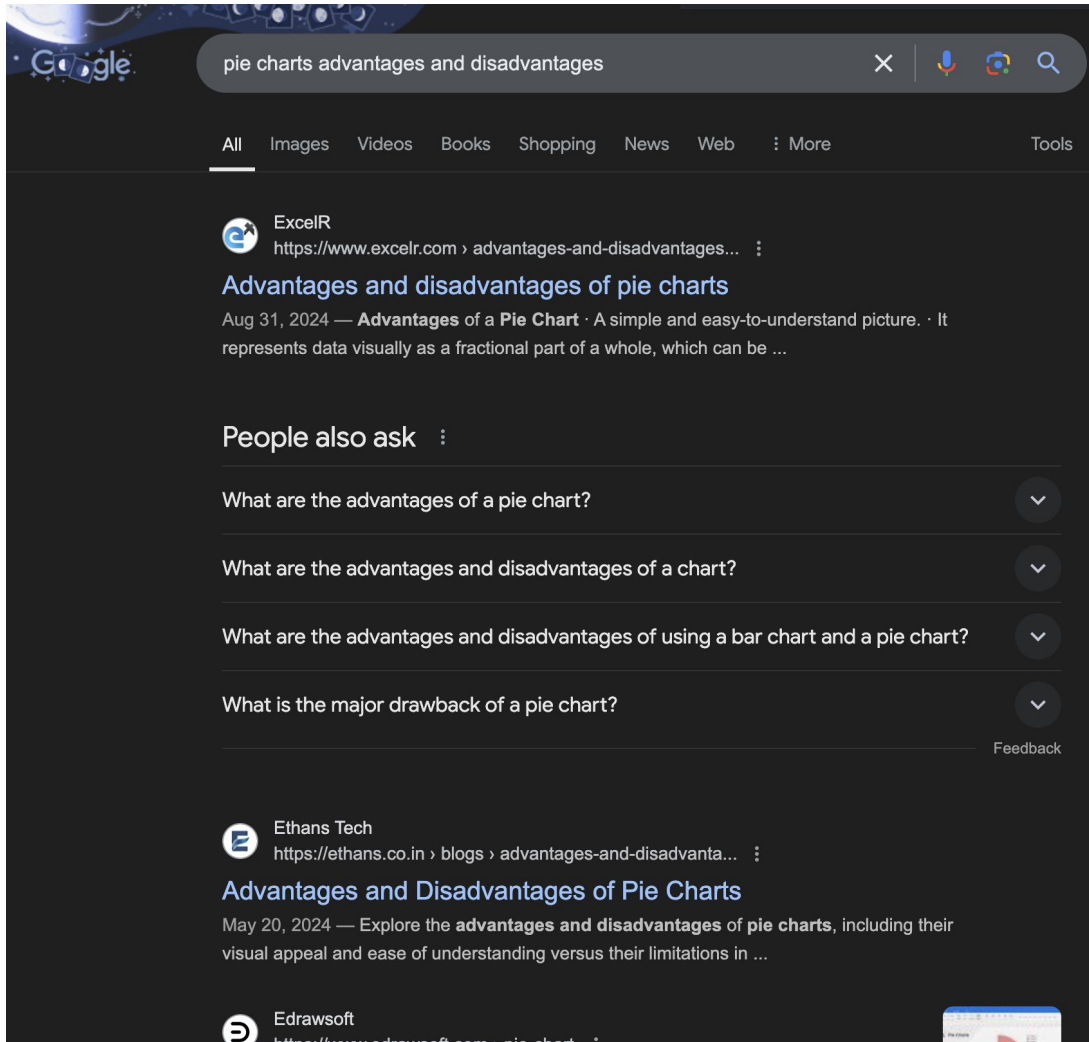
Specifies the starting point for lining up elements by angle
(0 means the starting point is at 3 o'clock)

```
labels = [ 'Frogs', 'Hogs', 'Dogs', 'Logs' ]
sizes = [ 15, 30, 45, 10 ]
colors = [ 'yellowgreen', 'gold', 'lightskyblue', 'lightcoral' ]
explode = ( 0, 0.1, 0, 0 )
```

```
# show graph
plt.pie(x = sizes,
        explode = explode,
        labels = labels,
        colors = colors,
        autopct = '%1.1f%%',
        shadow = True,
        startangle = 90)
```

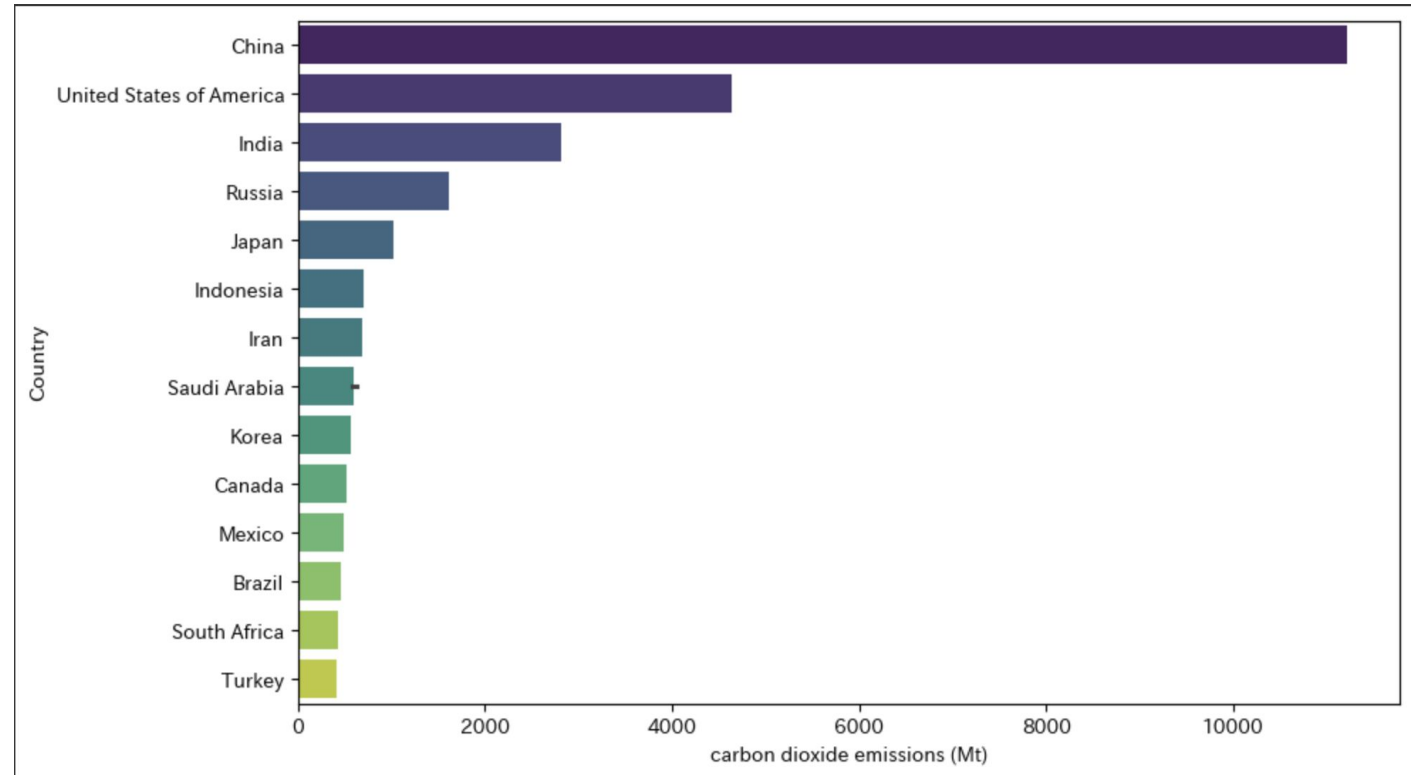
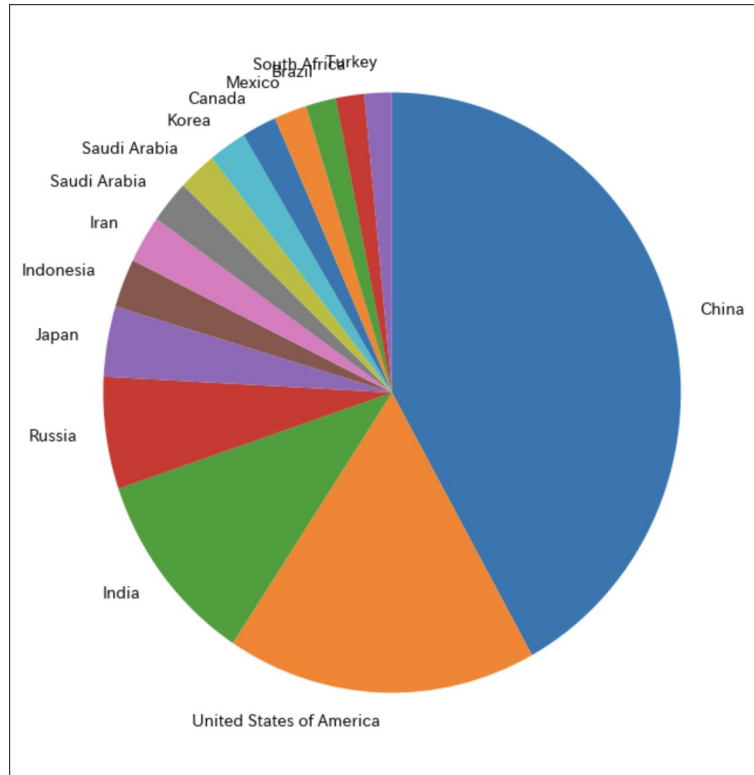


Pie charts need to be used with care



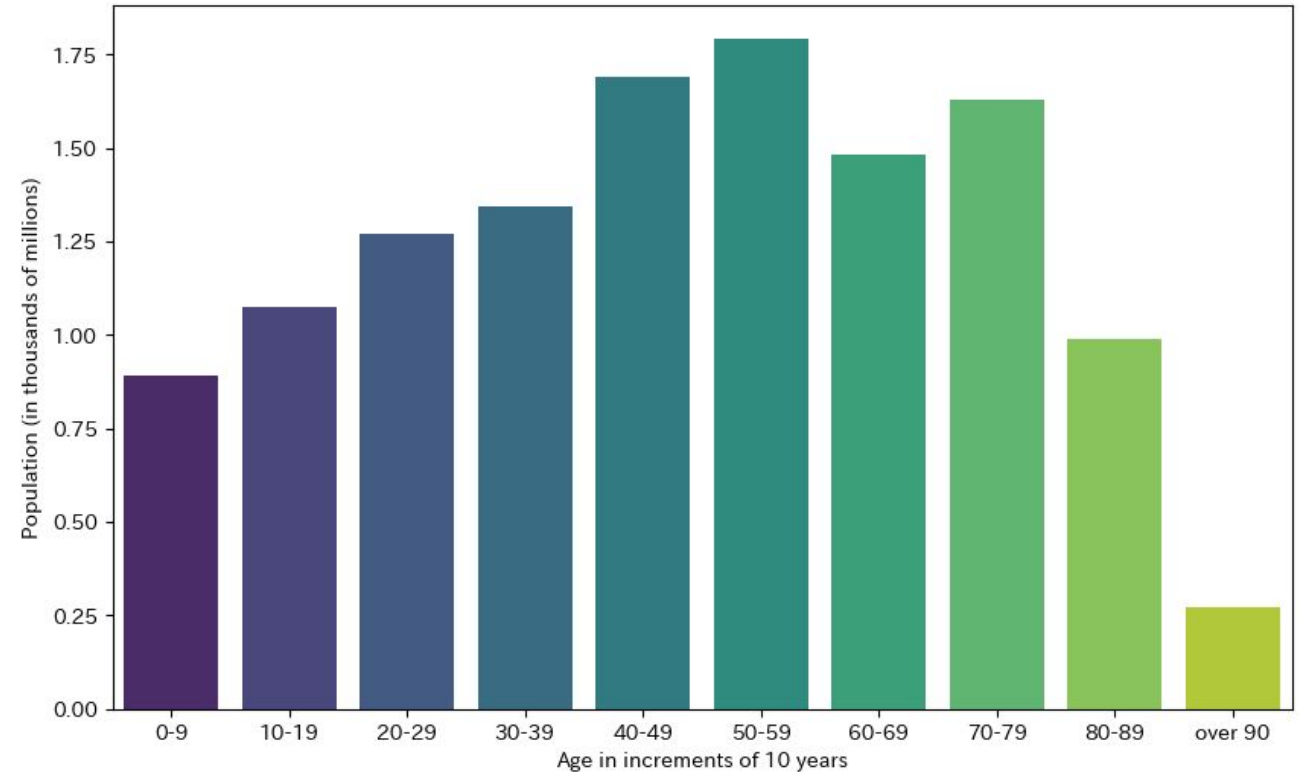
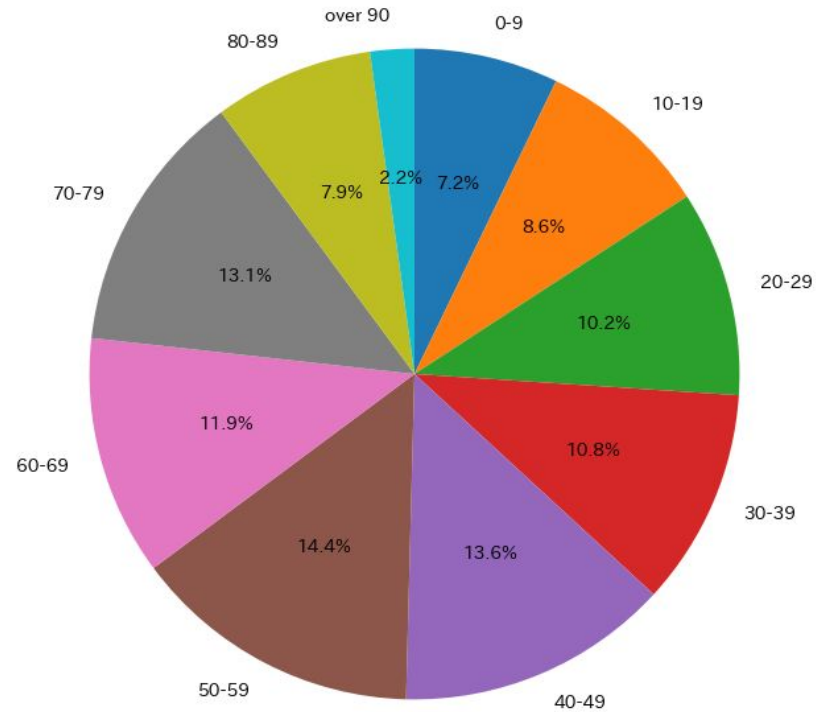
- Just keep in mind that there are pros and cons to using pie charts.

Pie charts need to be used with care



- Pie chart is easy to read when there is an element with a dominant share, but the same thing can be read from a bar chart
- Adjusting a pie chart often requires a lot of time

Pie charts need to be used with care

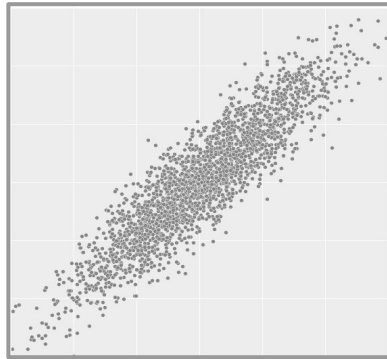


- Pie charts tend to look cluttered and hard to read if there isn't any dominant element
- Compared to bar charts, pie charts make it difficult to intuitively grasp differences among elements

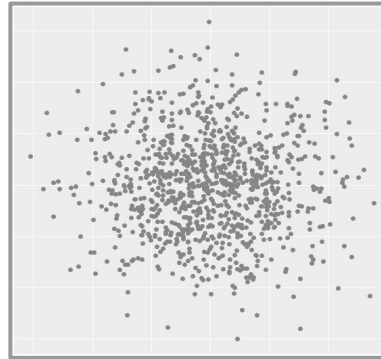
Properties of correlation coefficients



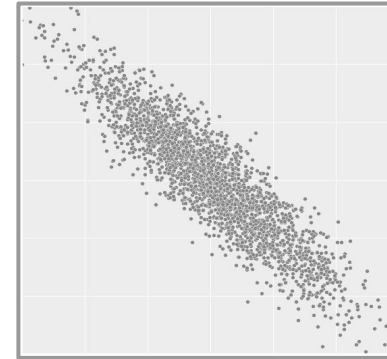
- Take values from -1 to 1.
 - Value closer to 1: strong positive correlations
 - Value closer to -1: strong negative correlations
 - The closer to 0, the weaker the correlation
- Coefficients expressing linkage such that a change in one value causes a change in the other
- Whether there is a causal relationship or not is not defined



Positive correlation

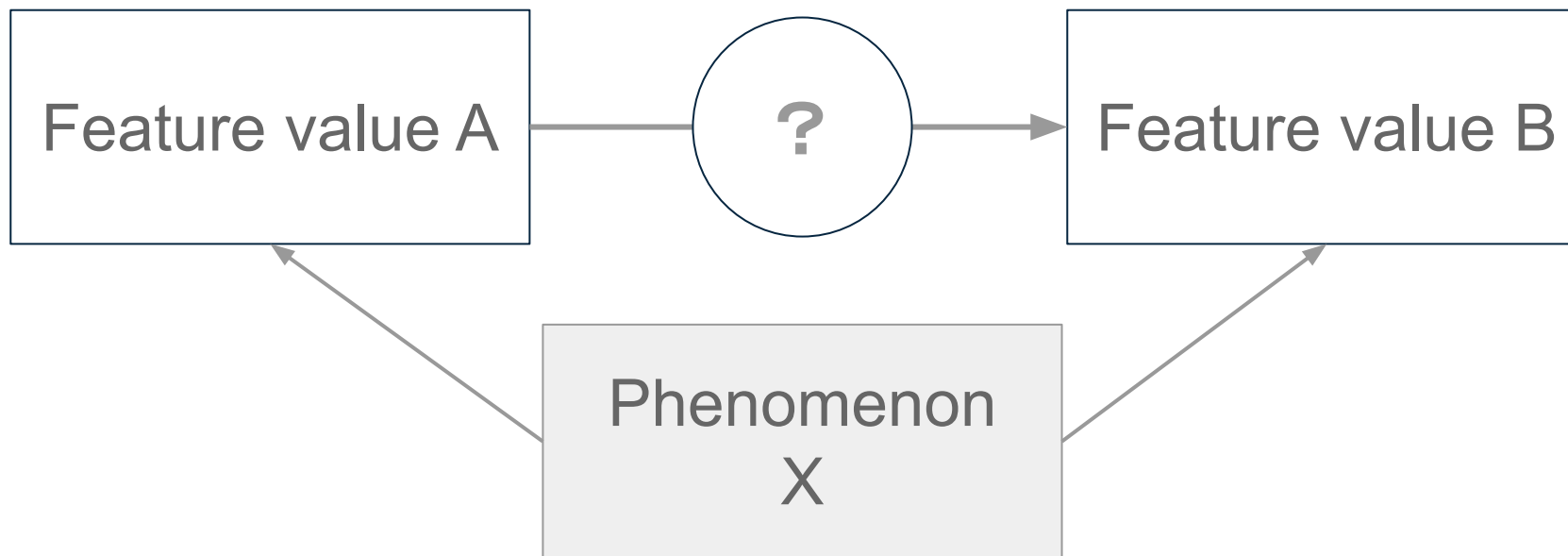


No correlation



Negative correlation

Correlation is not the same as causation.



- Even if there is a strong correlation between features A and B, other factors may be acting on both
- It can also be coincidental
- Causality cannot be easily obtained by coefficients, but requires careful consideration using domain knowledge

Summary: Types of Graphs

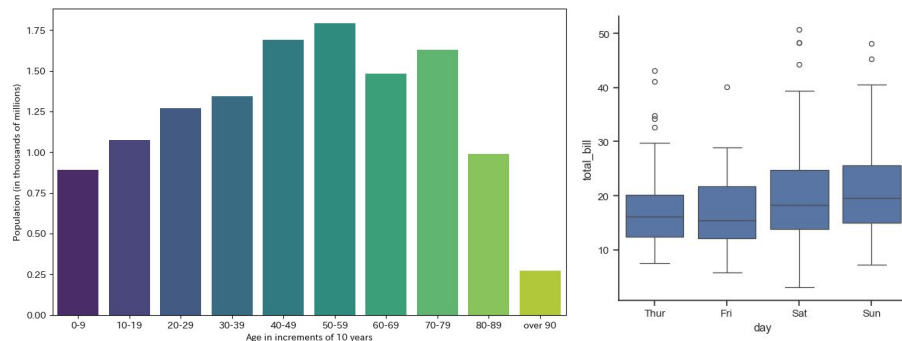


Quantitative data

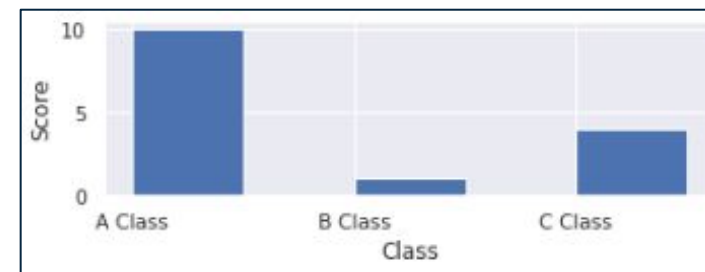
Categorical data

1 feature

Histogram / Box plot
plt.hist / plt.boxplot

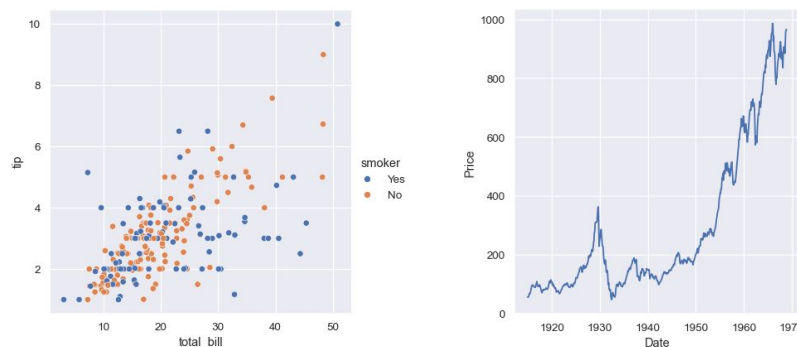


Bar plot / Pie chart
plt.bar / plt.pie

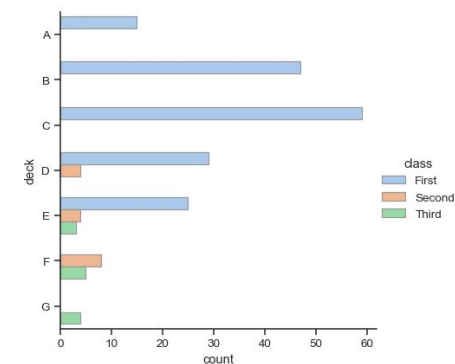


2 features

Scatter plot / Line plot / Heatmaps
plt.scatter / plt.plot / sns.heatmap



Side-by-side bar plot
plt.bar





Appendix | (Review) Create graphs in **a few lines** with matplotlib.pyplot

(1) Prepare the data to be drawn

(2) Specify at least 2 points: the data for each axis and the graph type

Function for **plotting** graph
`plt.plot()`

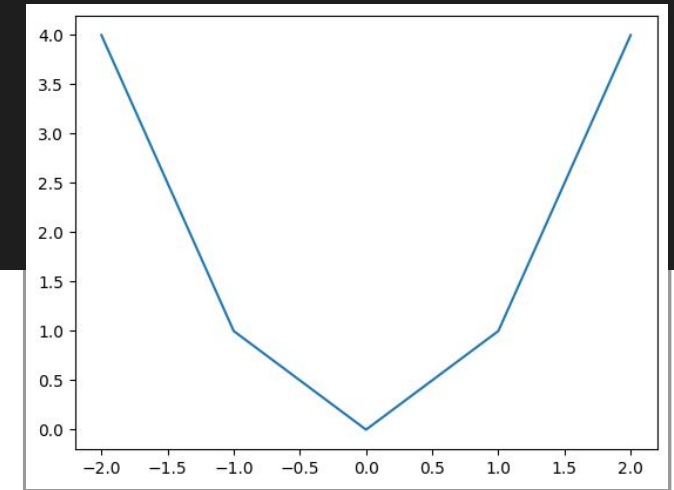
(3) Plot the graph

Function for **showing** graph
`plt.show()`

```
# Data to be displayed on each of the x- and y-axes
x = [-2, -1, 0, 1, 2]
y = [4, 1, 0, 1, 4]

# Draw graph
plt.plot(x, y, '-')

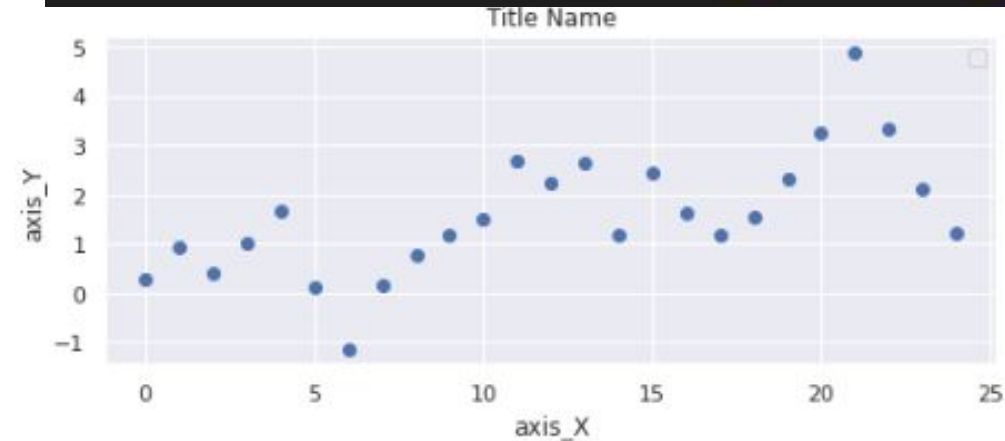
# Show graph
plt.show()
```



Minimally, this is all you need to draw a graph.
= **Making graphs in Python is easy**

- plt.plot (1) generates line graphs
- plt.scatter (2) generates scatter graphs
- In the case of method (2), various graphs can be used by replacing the “**scatter**” with the names of various graphs.

```
(1) plt.plot(x, y, "-")  
(2) plt.scatter(x, y)
```

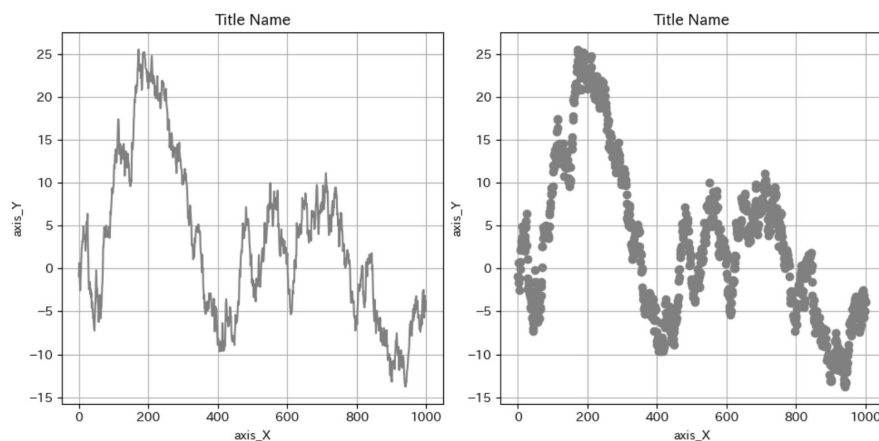


Appendix | Customizing graphs (1): changing the marker

You can easily change the markers of the graph by adjusting the third argument.

'-'	Line plot
'o'	Scatter plot

*See next page for more types.



```
# Suppose that data like this is contained in x and y
x = np.arange(1000)
y = np.random.randn(1000).cumsum( )
```

```
# Explained in the previous slide
plt.title('Title Name')
plt.xlabel('axis_X')
plt.ylabel('axis_Y')
```

```
# 3rd argument can be used to change the graph type
plt.plot(x,y, 'o')
```

```
plt.show( )
```

marker	symbol	description
"."		point
","		pixel
"o"		circle
"v"		triangle_down
"^"		triangle_up
"<"		triangle_left
">"		triangle_right
"1"		tri_down
"2"		tri_up
"3"		tri_left
"4"		tri_right
"8"		octagon
"s"		square
"p"		
"p"		

"*"		star
"h"		hexagon1
"H"		hexagon2
"_"		plus
"x"		x
"X"		x (filled)
"D"		diamond
"d"		thin_diamond
" "		vline
"_"		hline
0 (TICKLEFT)		tickleft
1 (TICKRIGHT)		tickright
2 (TICKUP)		tickup
3 (TICKDOWN)		tickdown
4 (CARETLEFT)		caretleft

8 (CARETLEFTBASE)		caretleft (centered at base)
9 (CARETRIGHTBASE)		caretright (centered at base)
10 (CARETUPBASE)		caretup (centered at base)
11 (CARETDOWNBASE)		caredown (centered at base)
"None", " " or ""		nothing
'\$...\$'		Render the string using mathtext. E.g "\$f\$" for marker showing the letter <i>f</i> .
verts		A list of (x, y) pairs used for Path vertices. The center of the marker is located at (0, 0) and the size is normalized, such that the created path is encapsulated inside the unit cell.
path		A <code>Path</code> instance.
(numsides, 0, angle)		A regular polygon with <code>numsides</code> sides, rotated by <code>angle</code> .
(numsides, 1, angle)		A star-like symbol with <code>numsides</code> sides,

The documentation provides a quick reference to the myriad of marker types available.
https://matplotlib.org/stable/api/markers_api.html

tated by

Appendix | Customizing graphs (2): Adding title and axis labels



(1) Add title using
plt.title()

```
# Explained in the previous slide  
x = [-1.5, 0.7, -0.4, -0.2, -0.3]  
y = [-1.3, -0.6, -0.2, 1.4, 0.4]  
plt.plot(x, y, 'o')
```

(1)

```
# Add graph title  
plt.title('Title Name')
```

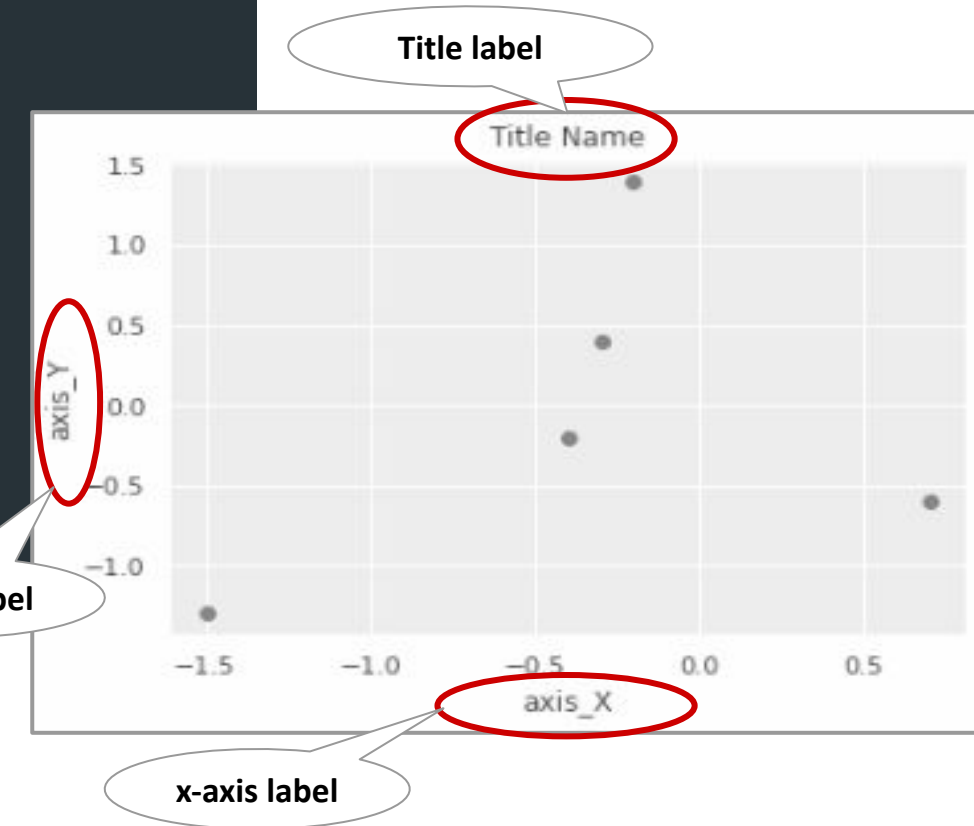
(2)

```
# Add x-axis label  
plt.xlabel('axis_X')  
  
# Add y-axis label  
plt.ylabel('axis_Y')
```

```
plt.show()
```

(2) Add label to x-axis and
y-axis using

plt.xlabel()
plt.ylabel()



Appendix | Customizing graphs (3): Adding legend and grid lines



(1) Specify graph label using argument: label

```
# Graph labels can be specified with the plot function argument [label=]  
plt.plot(x, y1, label='product1')  
plt.plot(x, y2, label='product2')
```

(1)

(2) Add legend using
plt.legend()

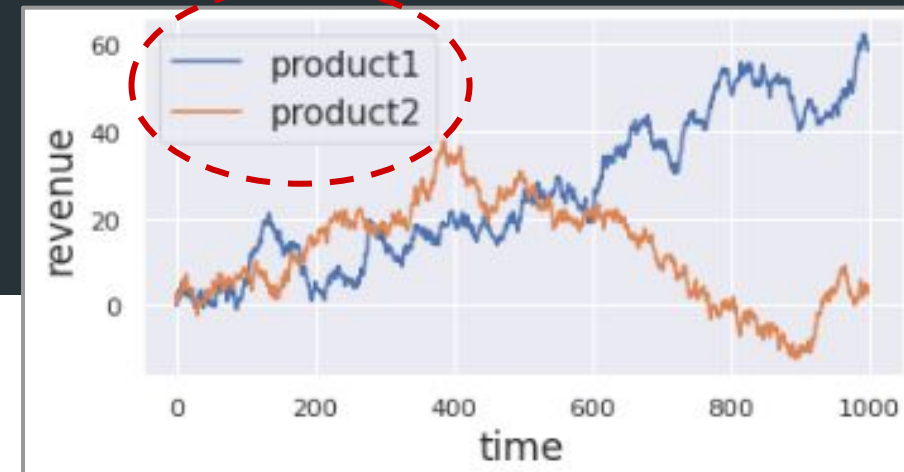
```
# Add legend  
plt.legend()
```

(2)

(3) Add grid lines using
plt.grid()
Pass boolean argument
(default: False)

```
plt.grid(True)
```

(3)



You can change the font size of each label by specifying **fontsize** as an argument.

In addition to font size, colors, typefaces, and other details can be adjusted.

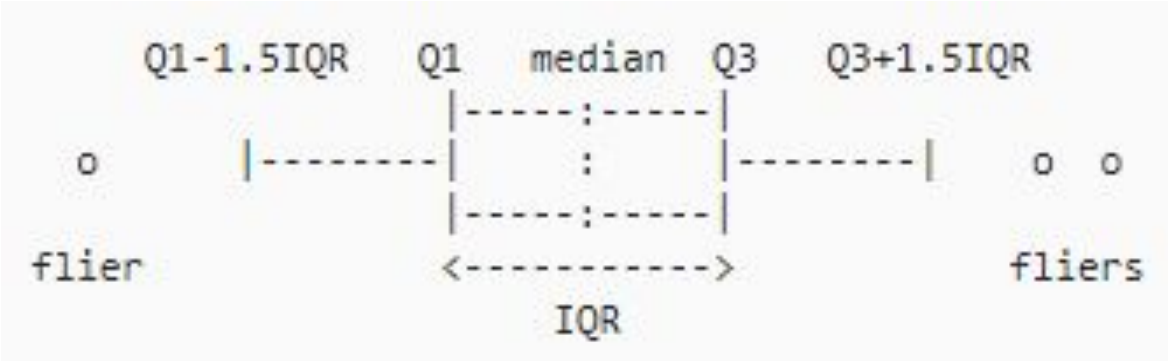
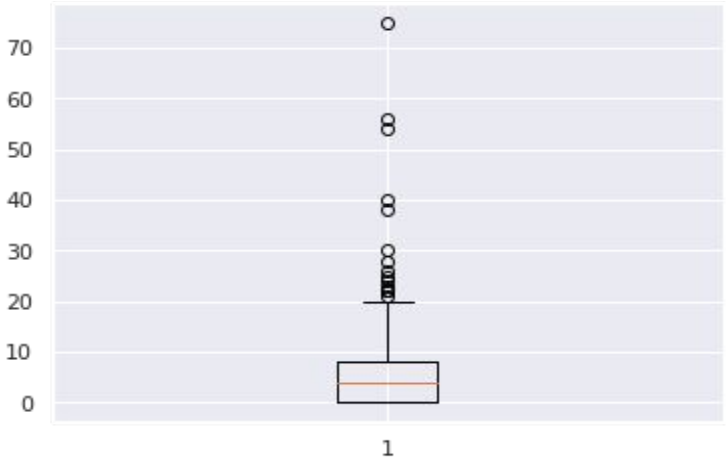
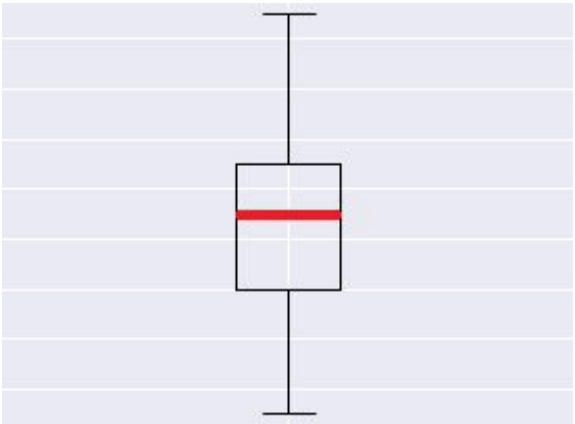
Learn how to look up details instead of memorizing them.

```
# Graph title
plt.title('Title', fontsize=20)
# x-axis label
plt.xlabel('X_label', fontsize=20)
# y-axis label
plt.ylabel('Y_label', fontsize=20)

# x-axis scale
plt.xticks(fontsize=20)
# y-axis scale
plt.yticks(fontsize=20)
# legend
plt.legend(fontsize=20)
```

function
about
scale

Bottom of box	1st quartile 25 percentile
Red line inside the box	2nd quartile 50 percentile Median
Top bottom of the box	3rd quartile 75 percentile
Edge of the whiskers	1.5 times tyhe Interquatile range (IQR)
Flyer point	outliers beyond 1.5 times the interquartile range (IQR)



Reference:
https://matplotlib.org/stable/api/as_gen/matplotlib.pyplot.boxplot.html

Use `plt.bar()` to create a **bar plot**

`plt.xticks()`, `plt.yticks()`

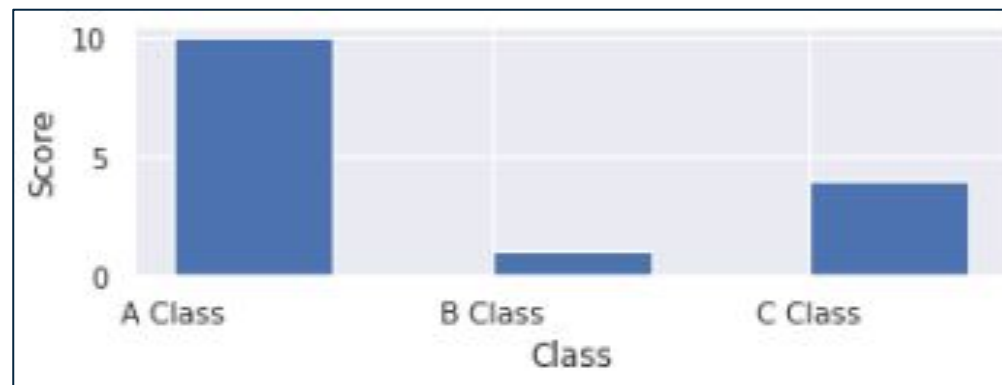
Specify parameters for **scale** on each of the x and y axes
Specify a list of values for scale position as the first argument and a list of scale labels as the second argument

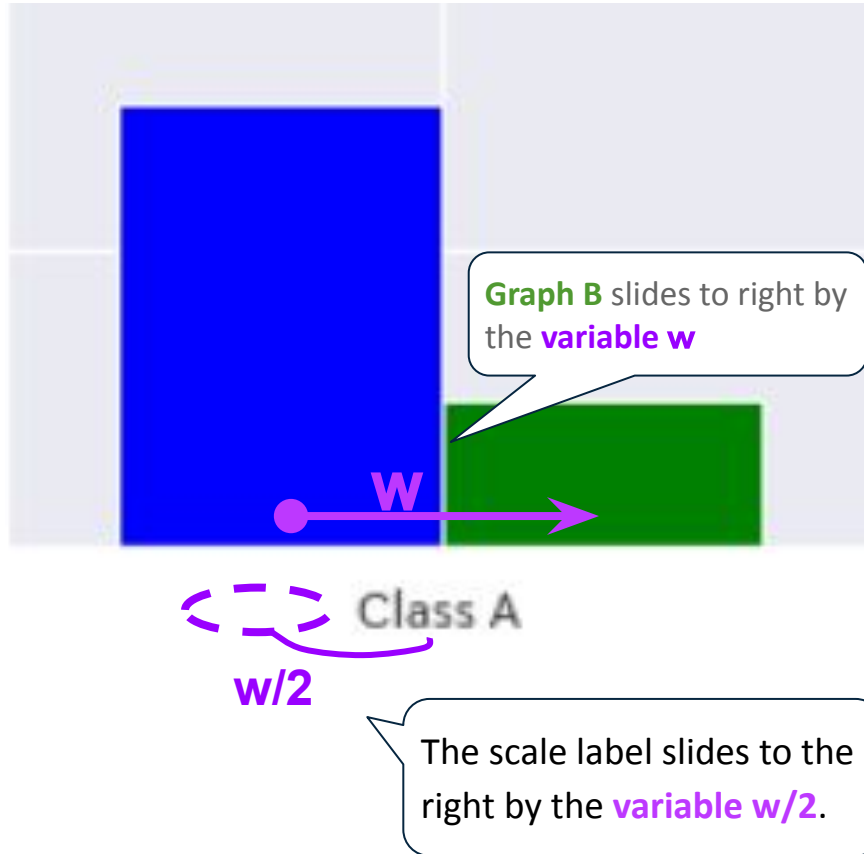
```
# data to be displayed  
x = [1, 2, 3]  
y = [10, 1, 4]
```

```
plt.bar(x, y, align='edge', width = 0.5)
```

```
# label for each bar graph
```

```
plt.xticks(x, ['A Class', 'B Class', 'C Class'])
```





$w = 0.4$

(1) Set "bar thickness" as variable w (to simplify the code)

(2) Plot graph A as usual

```
plt.bar( x, y1, width = w )
```

x x-axis position y1 y-axis position

(3) Adjust Graph B using variable w (Thickness of graph A) as x-axis position and width

```
plt.bar( x + w, y2, width = w )
```

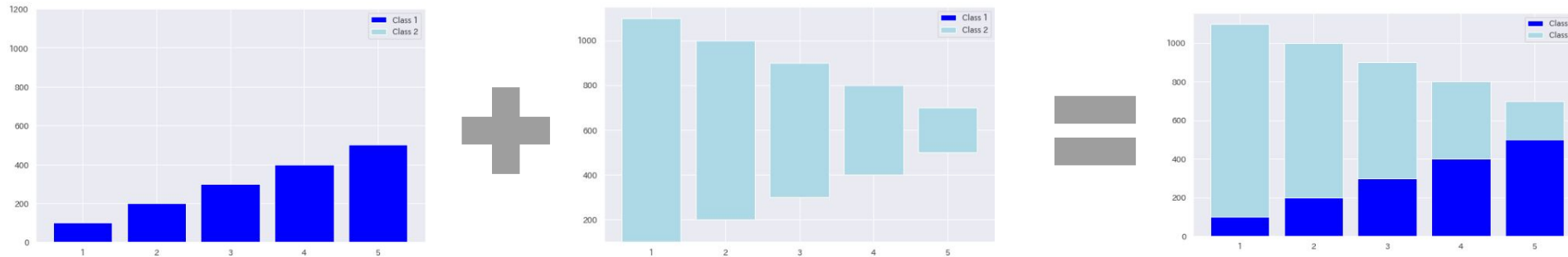
x + w x-axis position y2 y-axis position w Thickness of bar

(4) Adjust xticks by adding half of the variable w , ("thickness of graph A")

```
plt.xticks(x + w / 2, ['Class A', 'Class B'])
```

x + w / 2 x-axis position

Appendix | Bar Chart Applications: Stacked Bar Charts



```
p1 = plt.bar( x, height1,
```

x-axis position

height of bar graph

```
p2 = plt.bar( x, height2, bottom = height1,
```

x-axis position

height of bar graph

y-axis position

The value of "**bar height**" in p1 is directly specified as "**y-axis height**" p2.

argument "**bottom**"

Argument that specifies the height of the bar chart along the y-axis. The bottom of the bar is raised by the passed value.

How to write Matplotlib

```
plt.scatter(x = df['sepal_length'], y = df['petal_width'])
```

Specify x-axis data

Specify y-axis data

How to write seaborn

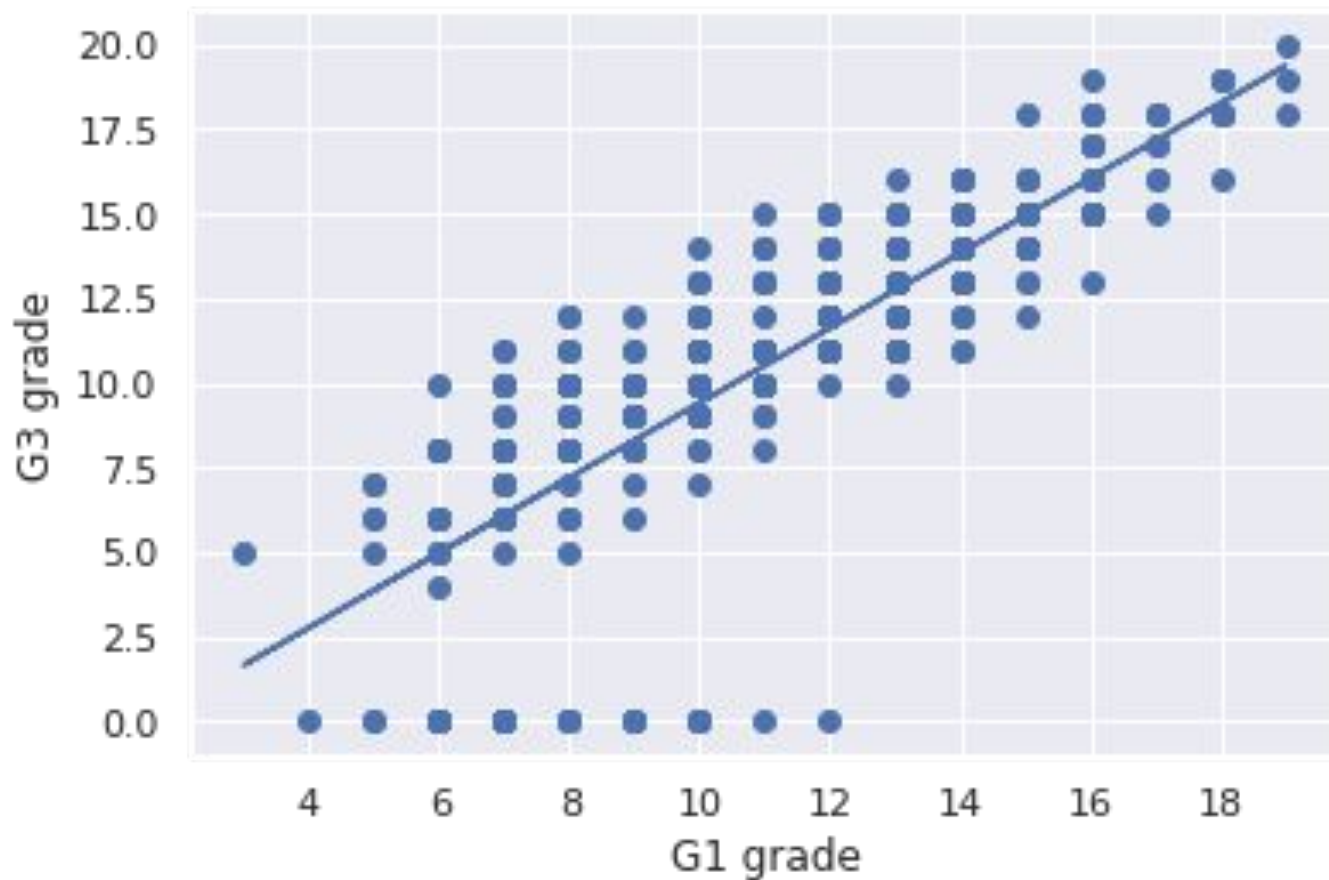
```
sns.scatterplot(data=df, x='sepal_length', y='petal_width')
```

Specify dataset

Specify x-axis
column name

Specify y-axis
column name

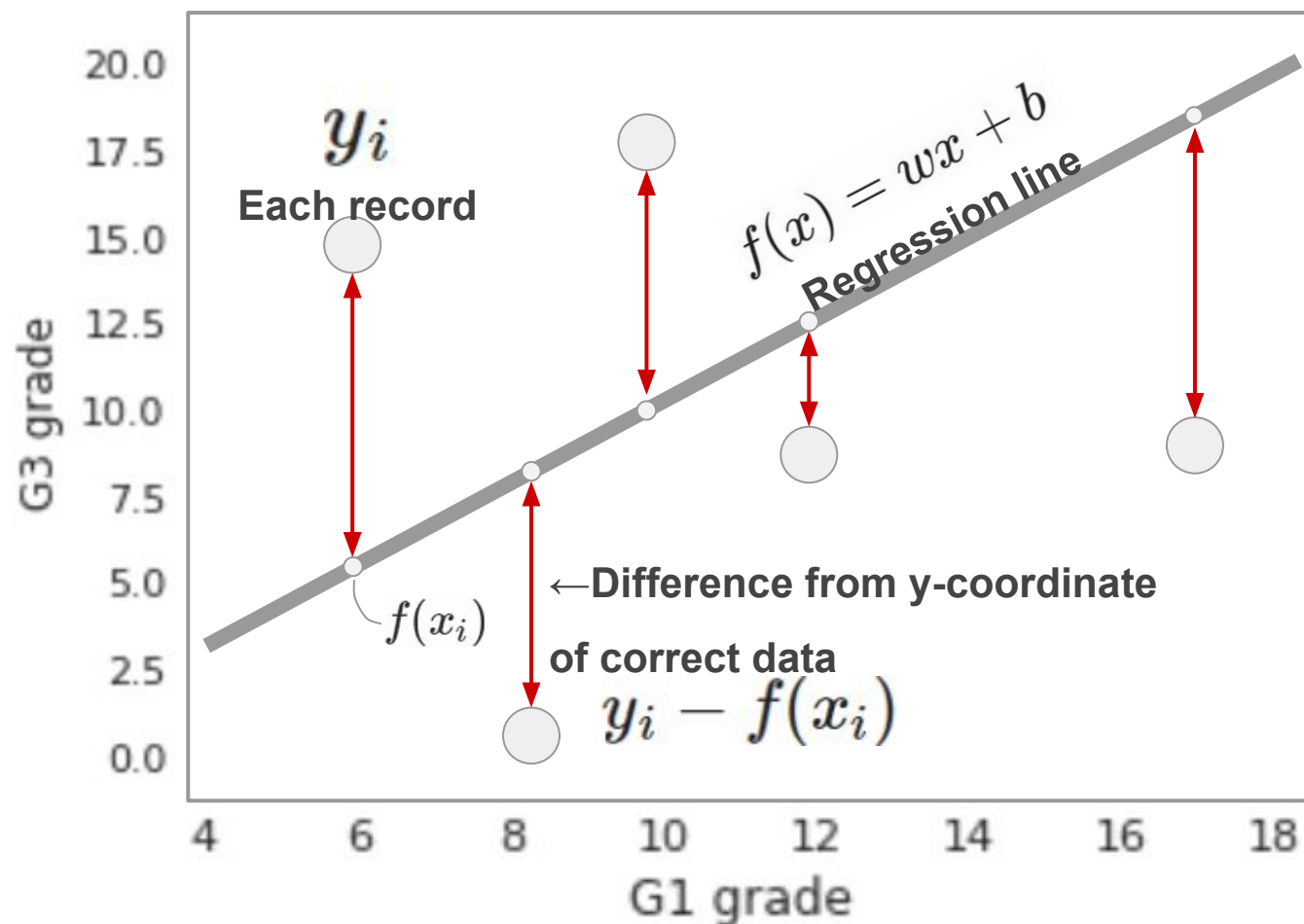
- Seaborn can be written in a similar way as matplotlib
- Seaborn can use “data” argument to specify the dataset and write only column names in the “x” and “y” arguments
 - Easier to use when using table data



$$y = wx + b$$

Target variable Explanatory variable

Coefficient Intercept



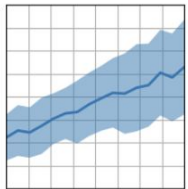
$$f(x) = wx + b$$

Find the coefficient w and the intercept b that minimize the error (automatic calculation)

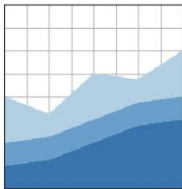
$$\sum_{i=1}^n (y_i - f(x_i))^2$$

Squared error

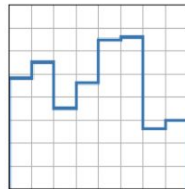
There are countless types of graphs,
so make sure to check out the documentation.



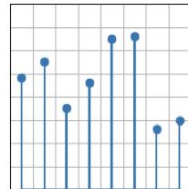
`fill_between(x, y1,
y2)`



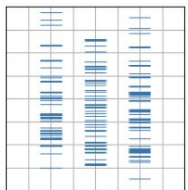
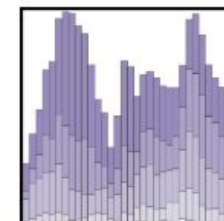
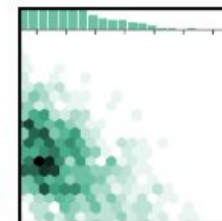
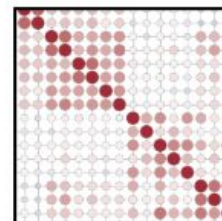
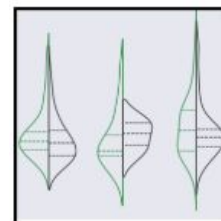
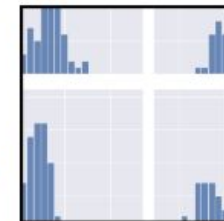
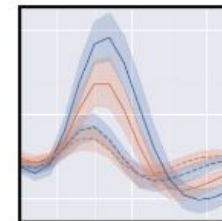
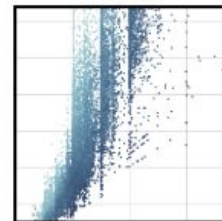
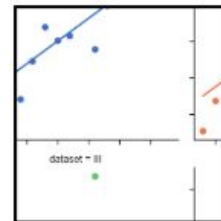
`stackplot(x, y)`



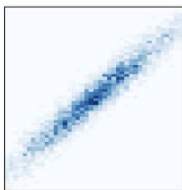
`stairs(values)`



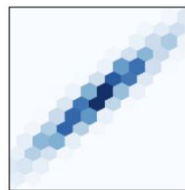
`stem(x, y)`



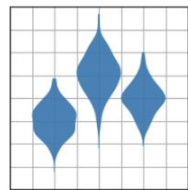
`eventplot(D)`



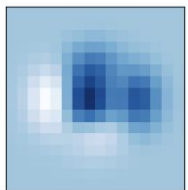
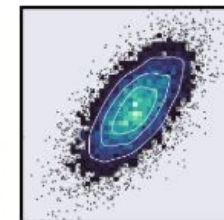
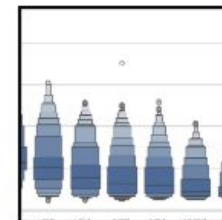
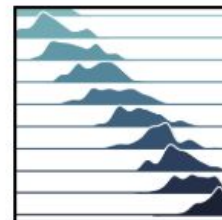
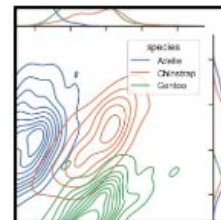
`hist2d(x, y)`



`hexbin(x, y, C)`



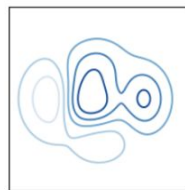
`violinplot(D)`



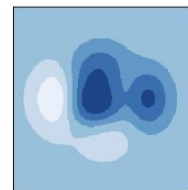
`imshow(Z)`



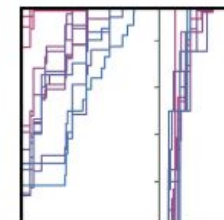
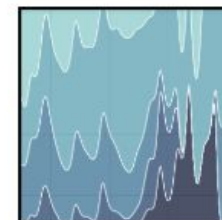
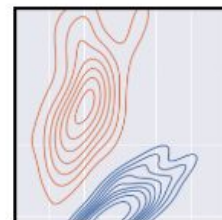
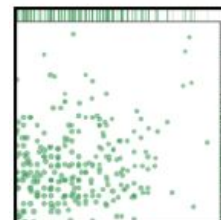
`pcolormesh(X, Y, Z)`



`contour(X, Y, Z)`



`contourf(X, Y, Z)`



Matplotlib

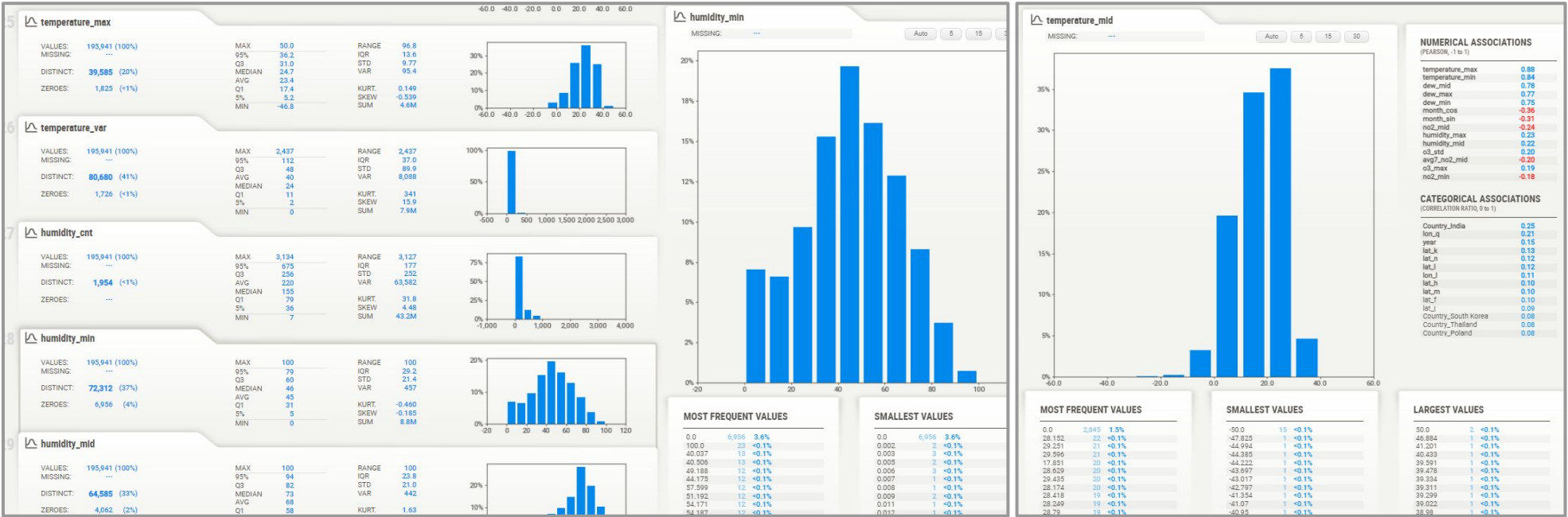
Seaborn

字幕エリア

アバターエリア



<https://github.com/fbdesignpro/sweetviz>

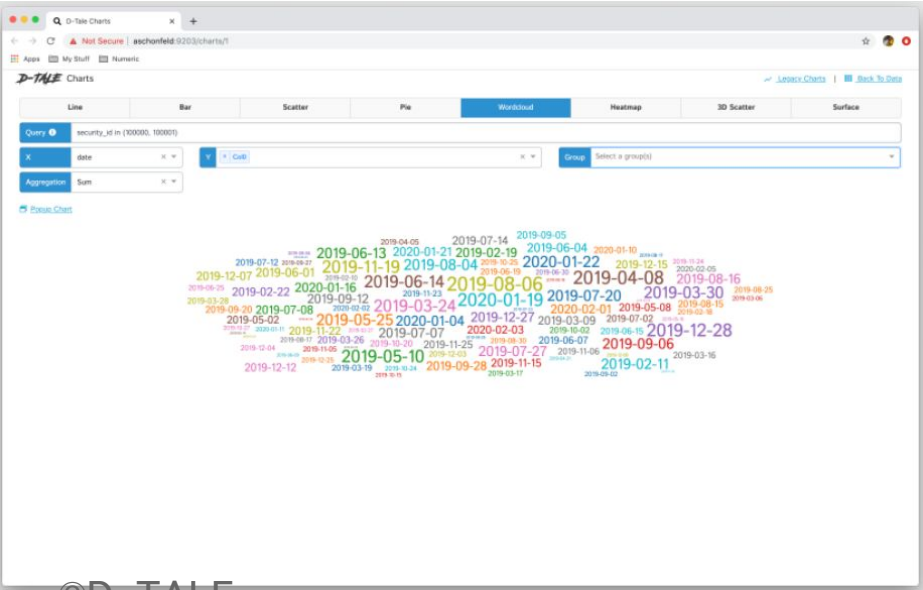


EDA Automation Library Example

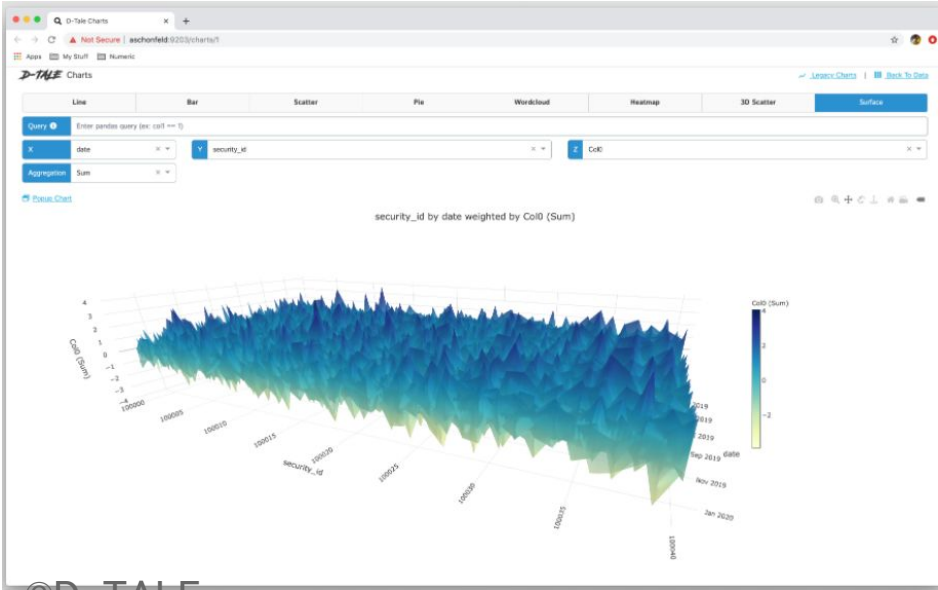


	date	security_id	int_val	Col0	Col1	Col2	Col3	Col4	Col5	Col6	Col7	Col8	Col9	Col10	Col11
0	2019-09-13	100000	13709556066	1.65	1.68	-0.09	0.68	2.04	1.40	-0.74	-0.15	-0.40	-0.07	0.16	0.90
1	2019-09-13	100001	32370834703	0.81	-0.68	-2.25	-0.58	1.05	0.95	-0.01	0.40	-0.63	0.24	-1.62	1.99
2	2018-09-13	100002	74359132292	0.27	1.39	0.00	-0.31	1.15	0.27	-1.45	0.28	0.04	0.00	0.61	0.61
3	2018-09-13	100003	3555606068	0.24	0.00	0.00	-0.81	0.00	0.85	-0.95	0.00	0.00	0.00	-1.74	-0.48
4	2018-09-13	100004	105740597	0.64	-1.38	0.00	1.00	0.00	0.44	-1.81	0.00	0.00	0.00	-0.02	1.05
5	2018-09-13	100005	3041943406	0.00	0.00	0.73	0.00	0.00	0.00	-0.85	0.00	0.00	0.00	0.41	1.03
6	2018-09-13	100006	9415021503	0.07	0.00	0.00	0.00	0.00	0.00	0.39	-1.16	0.00	0.00	1.76	1.11
7	2018-09-13	100007	9415021503	0.07	0.00	0.00	0.00	0.00	0.00	0.39	-1.16	0.00	0.00	1.76	1.11
8	2018-09-13	100008	7207483836	-0.03	0.68	-0.06	-0.58	0.00	1.13	0.00	0.09	-1.13	0.00	-0.07	-0.07
9	2018-09-13	100009	2405469457	0.78	0.28	0.66	0.88	0.71	0.57	-0.51	-0.29	-0.71	0.48	-1.07	0.41
10	2018-09-13	100010	5572953089	-0.24	-0.04	-0.29	1.63	-2.35	-0.19	0.35	-1.19	1.96	-1.02	0.43	1.40
11	2018-09-13	100011	79679700690	-0.41	0.34	-1.21	0.17	-0.94	-0.35	-1.54	-0.74	-0.81	-1.05	0.57	-0.20
12	2018-09-13	100012	35404502206	0.26	0.93	0.09	-0.11	0.77	-0.41	0.25	-1.92	1.05	-1.74	-0.58	-0.36

<https://github.com/man-group/dtale>



©D-TALE



©D-TALE